Project Information			
Group ID	07		**Semester** *(WS2?)*
1. Student Name	Viorel Tsigos		WS1
2. Student Name	Maximilian Peresunchak		WS1
3. Student Name	Nico Romano Reng		WS1
4. Student Name	Benjamin Möller		?
Project Title	Spatially Sharded 2D Tanks Multiplayer Game		

Project Description	We will develop a 2D multiplayer tank game in Rust with a shared, continuous arena divided into spatial sectors. Each sector is handled by a separate server, called a Shard Worker, which simulates the player controlled tanks and other entities in its designated area. Players send their input commands to the server responsible for their current location. An elected Matchmaster coordinates the servers, assigns sequence numbers to important game events, and monitors server availability. The goal is to allow new servers and players to discover an active instance while keeping the game running when a server fails.

Architectural Model	Hybrid client-server architecture with an inter-server control network: Client-Shard communication (UDP): Players send movement and action inputs to their assigned Shard Worker. The worker sends game-state updates back to its connected clients. Server-to-server communication (TCP): Shard Workers maintain connections with the Matchmaster and other workers. These connections are used for membership information, exchanging data about entities near sector boundaries, transferring players between sectors, and replicating important game events and state checkpoints. Concurrency: We intend to use Rust and Tokio to handle multiple client connections and server-to-server communication asynchronously.

Dynamic Discovery	UDP multicast for server and client discovery Server discovery: A new Shard Worker announces its address and intended sector through a multicast discovery message. The Matchmaster responds over TCP with the current cluster membership and sector assignments. Client discovery: A client sends a discovery request and receives the available Shard Worker endpoints needed to join the arena. Dynamic membership: New workers can join an existing cluster without restarting the running servers. The Matchmaster provides the current topology and assigns the new worker its sector together with the current game state.

Fault Tolerance	Dual-layer resilience covering crash failures and Byzantine coordinator behavior: Crash Detection & Heartbeats: Workers and the Matchmaster exchange TCP heartbeats every 500 ms in both directions. Three consecutive missed pings (1.5 s) trigger an automatic failure suspicion. Shard Worker Crash Recovery: Workers continuously mirror compact state checkpoints to an assigned backup neighbor. If a worker crashes, the Matchmaster reassigns its sector to the backup, which loads the checkpoint, replays recent committed events, and redirects connected clients. Client Disconnects: Authoritative worker drops the connection and despawns the tank for client timeouts. Byzantine Fault Detection for Coordination Log: We scope this only to the Matchmaster™s event sequencer. If a compromised or faulty Matchmaster issues contradictory sequence numbers or different event payloads for the same sequence slot to different workers, workers cross-validate signed event receipts. Upon detecting inconsistency, workers reject the proposal, mark the Matchmaster as Byzantine, and immediately force a leader election to replace it without stopping the simulations.

Voting	We intend to use the Bully election algorithm Each server has a unique numeric ID that determines its election priority. The servers maintain a membership table containing the known cluster members. An election starts when the cluster is initialized or a Shard Worker detects that the Matchmaster has failed. The worker starts an election by contacting servers with higher IDs. If no higher-priority server responds, it becomes the new Matchmaster. Otherwise, the higher-priority server takes over the election. After an election, the servers update their membership information and recognize the new Matchmaster as the cluster coordinator and event sequencer.

Reliable Ordered Multicast	Sequencer-based total ordering of important game events Player movement inputs can tolerate occasional packet loss, but events that change shared game state must be processed consistently across the relevant Shard Workers. Examples include player eliminations, power-up collection, match start and end events, and changes to the match score. The Matchmaster acts as the event sequencer. Shard Workers send important game events to it, and it assigns each event a monotonically increasing sequence number before distributing it to the workers. Workers buffer events that arrive out of order and apply them only when the next expected sequence number is available. Workers acknowledge received events over TCP. Missing acknowledgements trigger retransmission so that events are not silently lost. This ensures that participating workers apply important game events in the same order and avoid conflicting game-state updates.

System Architecture Design
Include a clear diagram of your system architecture (key components and connections between them).

